

第 8 章 备份与恢复

对于 DBA 来说，数据库的备份与恢复是一项最基本的操作与工作。在意外情况下（如服务器宕机、磁盘损坏、RAID 卡损坏等）要保证数据不丢失，或者是最小程度地丢失，每个 DBA 应该每时每刻关心所负责的数据库备份情况。

本章主要介绍对 InnoDB 存储引擎的备份，应该知道 MySQL 数据库提供的大多数工具（如 mysqldump、ibbackup、replication）都能很好地完成备份的工作，当然也可以通过第三方的一些工具来完成，如 xtrabacup、LVM 快照备份等。DBA 应该根据自己的业务要求，设计出损失最小、对于数据库影响最小的备份策略。

8.1 备份与恢复概述

可以根据不同的类型来划分备份的方法。根据备份的方法不同可以将备份分为：

- ☐ Hot Backup（热备）
- ☐ Cold Backup（冷备）
- ☐ Warm Backup（温备）

Hot Backup 是指数据库运行中直接备份，对正在运行的数据库操作没有任何的影响。这种方式在 MySQL 官方手册中称为 Online Backup（在线备份）。Cold Backup 是指备份操作是在数据库停止的情况下，这种备份最为简单，一般只需要复制相关的数据库物理文件即可。这种方式在 MySQL 官方手册中称为 Offline Backup（离线备份）。Warm Backup 备份同样是在数据库运行中进行的，但是会对当前数据库的操作有所影响，如加一个全局读锁以保证备份数据的一致性。

按照备份后文件的内容，备份又可以分为：

- ☐ 逻辑备份
- ☐ 裸文件备份

在 MySQL 数据库中，逻辑备份是指备份出的文件内容是可读的，一般是文本

文件。内容一般是由一条条 SQL 语句，或者是表内实际数据组成。如 `mysqldump` 和 `SELECT*INTO OUTFILE` 的方法。这类方法的好处是可以观察导出文件的内容，一般适用于数据库的升级、迁移等工作。但其缺点是恢复所需要的时间往往较长。

裸文件备份是指复制数据库的物理文件，既可以是在数据库运行中的复制（如 `ibbackup`、`xtrabackup` 这类工具），也可以是在数据库停止运行时直接的数据文件复制。这类备份的恢复时间往往较逻辑备份短很多。

若按照备份数据库的内容来分，备份又可以分为：

- ☐ 完全备份
- ☐ 增量备份
- ☐ 日志备份

完全备份是指对数据库进行一个完整的备份。增量备份是指在上次完全备份的基础上，对于更改的数据进行备份。日志备份主要是指对 MySQL 数据库二进制日志的备份，通过对一个完全备份进行二进制日志的重做（`replay`）来完成数据库的 `point-in-time` 的恢复工作。MySQL 数据库复制（`replication`）的原理就是异步实时地将二进制日志重做传送并应用到从（`slave/standby`）数据库。

对于 MySQL 数据库来说，官方没有提供真正的增量备份的方法，大部分是通过二进制日志完成增量备份的工作。这种备份较之真正的增量备份来说，效率还是很低的。假设有一个 100GB 的数据库，要通过二进制日志完成备份，可能同一个页需要执行多次的 SQL 语句完成重做的工作。但是对于真正的增量备份来说，只需要记录当前每页最后的检查点的 LSN，如果大于之前全备时的 LSN，则备份该页，否则不用备份，这大大加快了备份的速度和恢复的时间，同时这也是 `xtrabackup` 工具增量备份的原理。

此外还需要理解数据库备份的一致性，这种备份要求在备份的时候数据在这一时间点上是一致的。举例来说，在一个网络游戏中有一个玩家购买了道具，这个事务的过程是：先扣除相应的金钱，然后向其装备表中插入道具，确保扣费和得到道具是互相一致的。否则，在恢复时，可能出现金钱被扣除了而装备丢失的问题。

对于 InnoDB 存储引擎来说，因为其支持 MVCC 功能，因此实现一致的备份比较简单。用户可以先开启一个事务，然后导出一组相关的表，最后提交。当然用户的事务隔离级别必须设置为 `REPEATABLE READ`，这样的做法就可以给出一个完美的一致性备份。然而这个方法的前提是需要用户正确地设计应用程序。对于上述的购买道具的过

程，不可以分为两个事务来完成，如一个完成扣费，一个完成道具的购买。若备份这时发生在这两者之间，则由于逻辑设计的问题，导致备份出的数据依然不是一致的。

对于 mysqldump 备份工具来说，可以通过添加 --single-transaction 选项获得 InnoDB 存储引擎的一致性备份，原理和之前所说的相同。需要了解的是，这时的备份是在一个执行时间很长的事务中完成的。另外，对于 InnoDB 存储引擎的备份，务必加上 --single-transaction 的选项（虽然是 mysqldump 的一个可选选项，但是我找不出任何不加的理由）。

同时我建议每个公司要根据自己的备份策略编写一个备份的应用程序，这个程序可以方便地设置备份的方法及监控备份的结果，并且通过第三方接口实时地通知 DBA，这样才能真正地做到 24×7 的备份监控。久游网开发过一套 DAO（Database Admin Online）系统，这套系统完全由 DBA 开发完成，整个平台用 Python 语言编写，Web 操作界面采用 Django。通过这个系统 DBA 可以方便地对几百台 MySQL 数据库服务器进行备份，同时查看备份完成后备份文件的状态。之后 DBA 又对其进行了扩展，不仅可以完成备份的工作，也可以实时监控数据库的状态、系统的状态和硬件的状态，当发生问题时，通过飞信接口在第一时间以短信的方式告知 DBA。

最后，任何时候都需要做好远程异地备份，也就是容灾的防范。只是同一机房的两台服务器的备份是远远不够的。我曾经遇到的情况是，公司在 2008 年的汶川地震中发生一个机房可能被淹的情况，这时远程异地备份显得就至关重要了。

8.2 冷备

对于 InnoDB 存储引擎的冷备非常简单，只需要备份 MySQL 数据库的 frm 文件，共享表空间文件，独立表空间文件 (*.ibd)，重做日志文件。另外建议定期备份 MySQL 数据库的配置文件 my.cnf，这样有利于恢复的操作。

通常 DBA 会写一个脚本来进行冷备的操作，DBA 可能还会对备份完的数据库进行打包和压缩，这都并不是难事。关键在于不要遗漏原本需要备份的物理文件，如共享表空间和重做日志文件，少了这些文件可能数据库都无法启动。另外一种经常发生的情况是由于磁盘空间已满而导致的备份失败，DBA 可能习惯性地认为运行脚本的备份是没有问题的，少了检验的机制。

正如前面所说的，在同一台机器上对数据库进行冷备是远远不够的，至少还需要将

本地产生的备份存放一台远程的服务器中，确保不会因为本地数据库宕机而影响备份文件的使用。

冷备的优点是：

- ☐ 备份简单，只要复制相关文件即可。
- ☐ 备份文件易于在不同操作系统，不同 MySQL 版本上进行恢复。
- ☐ 恢复相当简单，只需要把文件恢复到指定位置即可。
- ☐ 恢复速度快，不需要执行任何 SQL 语句，也不需要重建索引。

冷备的缺点是：

- ☐ InnoDB 存储引擎冷备的文件通常比逻辑文件大很多，因为表空间中存放着很多其他的数据，如 undo 段，插入缓冲等信息。
- ☐ 冷备也不总是可以轻易地跨平台。操作系统、MySQL 的版本、文件大小写敏感和浮点数格式都会成为问题。

8.3 逻辑备份

8.3.1 mysqldump

mysqldump 备份工具最初由 Igor Romanenko 编写完成，通常用来完成转存（dump）数据库的备份及不同数据库之间的移植，如从 MySQL 低版本数据库升级到 MySQL 高版本数据库，又或者从 MySQL 数据库移植到 Oracle、Microsoft SQL Server 数据库等。

mysqldump 的语法如下：

```
shell>mysqldump [arguments] >file_name
```

如果想要备份所有的数据库，可以使用 --all-databases 选项：

```
shell>mysqldump --all-databases >dump.sql
```

如果想要备份指定的数据库，可以使用 --databases 选项：

```
shell>mysqldump --databases db1 db2 db3 >dump.sql
```

如果想要对 test 这个架构进行备份，可以使用如下语句：

```
[root@xen-server ~]# mysqldump --single-transaction test >test_backup.sql
```

上述操作产生了一个对 test 架构的备份, 使用 --single-transaction 选项来保证备份的一致性。备份出的 test_backup.sql 是文本文件, 通过文本查看命令 cat 就可以得到文件的内容:

```
[root@xen-server ~]# cat test_backup.sql
-- MySQL dump 10.13  Distrib 5.5.1-m2, for unknown-linux-gnu (x86_64)
--
-- Host: localhost    Database: test
--
-----
-- Server version      5.5.1-m2-log
--
.....
--
-- Table structure for table 'a'
--

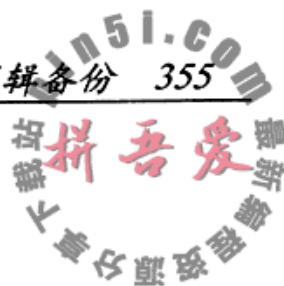
DROP TABLE IF EXISTS 'a';
/*!40101 SET @saved_cs_client      = @@character_set_client */;
/*!40101 SET character_set_client = utf8 */;
CREATE TABLE 'a' (
  'b' int(11) NOT NULL DEFAULT '0',
  PRIMARY KEY ('b')
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
/*!40101 SET character_set_client = @saved_cs_client */;

--
-- Dumping data for table 'a'
--

LOCK TABLES 'a' WRITE;
/*!40000 ALTER TABLE 'a' DISABLE KEYS */;
INSERT INTO 'a' VALUES (1),(2),(4),(5);
/*!40000 ALTER TABLE 'a' ENABLE KEYS */;
UNLOCK TABLES;

--
-- Table structure for table 'z'
--

DROP TABLE IF EXISTS 'z';
/*!40101 SET @saved_cs_client      = @@character_set_client */;
/*!40101 SET character_set_client = utf8 */;
CREATE TABLE 'z' (
```

```
'a' int(11) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
/*!40101 SET character_set_client = @saved_cs_client */;

--
-- Dumping data for table 'z'
--

LOCK TABLES 'z' WRITE;
/*!40000 ALTER TABLE 'z' DISABLE KEYS */;
INSERT INTO 'z' VALUES (1),(1);
/*!40000 ALTER TABLE 'z' ENABLE KEYS */;
UNLOCK TABLES;

.....

-- Dump completed on 2010-08-03 13:36:17
```

可以看到，备份出的文件内容就是表结构和数据，所有这些都是用 SQL 语句方式表示。文件开始和结束的注释部分是用来设置 MySQL 数据库的各项参数，一般用来使还原工作更有效和准确地进行。之后的部分先是 CREATE TABLE 语句，接着就是 INSERT 的 SQL 语句了。

mysqldump 的参数选项很多，可以通过使用 mysqldump--help 命令来查看所有的参数，有些参数有缩写形式，如 --lock-tables 的缩写形式 -l。这里列举一些比较重要的参数。

- ☐ --single-transaction：在备份开始前，先执行 START TRANSACTION 命令，以此来获得备份的一致性，当前该参数只对 InnoDB 存储引擎有效。当启用该参数并进行备份时，确保没有其他任何的 DDL 语句执行，因为一致性读并不能隔离 DDL 操作。
- ☐ --lock-tables (-l)：在备份中，依次锁住每个架构下的所有表。一般用于 MyISAM 存储引擎，当备份时只能对数据库进行读取操作，不过备份依然可以保证一致性。对于 InnoDB 存储引擎，不需要使用该参数，用 --single-transaction 即可。并且 --lock-tables 和 --single-transaction 是互斥（exclusive）的，不能同时使用。如果用户的 MySQL 数据库中，既有 MyISAM 存储引擎的表，又有 InnoDB 存储引擎的表，那么这时用户的选择只有 --lock-tables 了。此外，正如前面所说的那样，--lock-tables 选项是依次对每个架构中的表上锁的，因此只能保证每个架构下



表备份的一致性，而不能保证所有架构下表的一致性。

- ❑ **--lock-all-tables (-x)**: 在备份过程中，对所有架构中的所有表上锁。这个可以避免之前说的 **--lock-tables** 参数不能同时锁住所有表的问题。
- ❑ **--add-drop-database**: 在 **CREATE DATABASE** 前先运行 **DROP DATABASE**。这个参数需要和 **--all-databases** 或者 **--databases** 选项一起使用。在默认情况下，导出的文本文件中并不会会有 **CREATE DATABASE**，除非指定了这个参数，因此可能会看到如下的内容：

```
[root@xen-server ~]# mysqldump --single-transaction --add-drop-database
--databases test >test_backup.sql
[root@xen-server ~]# cat test_backup.sql
-- MySQL dump 10.13  Distrib 5.5.1-m2, for unknown-linux-gnu (x86_64)
.....
--
-- Current Database: 'test'
--

/*!40000 DROP DATABASE IF EXISTS 'test'*/;

CREATE DATABASE /*!32312 IF NOT EXISTS*/ 'test' /*!40100 DEFAULT CHARACTER SET
latin1 */;

USE 'test';
.....
```

- ❑ **--master-data [=value]**: 通过该参数产生的备份转存文件主要用来建立一个 replication。当 value 的值为 1 时，转存文件中记录 **CHANGE MASTER** 语句。当 value 的值为 2 时，**CHANGE MASTER** 语句被写出 SQL 注释。在默认情况下，value 的值为空。当 value 值为 1 时，在备份文件中会看到：

```
[root@xen-server ~]# mysqldump --single-transaction --add-drop-database
--master-data=1 --databases test >test_backup.sql
[root@xen-server ~]# cat test_backup.sql
-- MySQL dump 10.13  Distrib 5.5.1-m2, for unknown-linux-gnu (x86_64)
--
-- Host: localhost      Database: test
-- -----
-- Server version      5.5.1-m2-log
.....
```



```
--
-- Position to start replication or point-in-time recovery from
--
```

```
CHANGE MASTER TO MASTER_LOG_FILE='xen-server-bin.000006', MASTER_LOG_POS=8095;
.....
```

当 value 为 2 时，在备份文件中会看到 CHANGE MASTER 语句被注释了：

```
[root@xen-server ~]# mysqldump --single-transaction --add-drop-database
--master-data=2 --databases test >test_backup.sql
[root@xen-server ~]# cat test_backup.sql
-- MySQL dump 10.13  Distrib 5.5.1-m2, for unknown-linux-gnu (x86_64)
--
-- Host: localhost      Database: test
--
-----
-- Server version      5.5.1-m2-log
.....
--
-- Position to start replication or point-in-time recovery from
--
--
.....
```

- ☐ --master-data 会自动忽略 --lock-tables 选项。如果没有使用 --single-transaction 选项，则会自动使用 --lock-all-tables 选项。
- ☐ --events (-E)：备份事件调度器。
- ☐ --routines (-R)：备份存储过程和函数。
- ☐ --triggers：备份触发器。
- ☐ --hex-blob：将 BINARY、VARBINARY、BLOB 和 BIT 列类型备份为十六进制的格式。mysqldump 导出的文件一般是文本文件，但是如果导出的数据中有上述这些类型，在文本文件模式下可能有些字符不可见，若添加 --hex-blob 选项，结果会以十六进制的方式显示，如：

```
[root@xen-server ~]# mysqldump --single-transaction --add-drop-database
--master-data=2 --no-autocommit --databases test3 > test3_backup.sql
[root@xen-server ~]# cat test3_backup.sql
-- MySQL dump 10.13  Distrib 5.5.1-m2, for unknown-linux-gnu (x86_64)
--
```



```
-- Host: localhost      Database: test3
-- -----
-- Server version      5.5.1-m2-log
.....
LOCK TABLES 'a' WRITE;
/*!40000 ALTER TABLE 'a' DISABLE KEYS */;
setautocommit=0;
INSERT INTO 'a' VALUES (0x61000000000000000000);
/*!40000 ALTER TABLE 'a' ENABLE KEYS */;
UNLOCK TABLES;
```

可以看到，这里用 0x61000000000000000000 的十六进制的格式来导出数据。

❑ **--tab=path (-T path)**：产生 TAB 分割的数据文件。对于每张表，mysqldump 创建一个包含 CREATE TABLE 语句的 table_name.sql 文件，和包含数据的 tbl_name.txt 文件。可以使用 --fields-terminated-by=..., --fields-enclosed-by=..., --fields-optionally-enclosed-by=..., --fields-escaped-by=..., --lines-terminated-by=... 来改变默认的分割符、换行符等。如：

```
[root@xen-server test]# mysqldump --single-transaction --add-drop-database
--tab="/usr/local/mysql/data/test" test
[root@xen-server test]# ls -lh
total 244K
-rw-rw---- 1 mysql mysql 8.4K Jul 21 16:02 a.frm
-rw-rw---- 1 mysql mysql 96K Jul 22 17:18 a.ibd
-rw-r--r-- 1 root root 1.3K Aug 3 15:36 a.sql
-rw-rw-rw- 1 mysql mysql 8 Aug 3 15:36 a.txt
-rw-rw---- 1 mysql mysql 65 Jul 17 15:54 db.opt
-rw-rw---- 1 mysql mysql 8.4K Aug 2 17:22 z.frm
-rw-rw---- 1 mysql mysql 96K Aug 2 17:22 z.ibd
-rw-r--r-- 1 root root 1.3K Aug 3 15:36 z.sql
-rw-rw-rw- 1 mysql mysql 4 Aug 3 15:36 z.txt
-----
-- Server version      5.5.1-m2-log

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8 */;
/*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
/*!40103 SET TIME_ZONE='+00:00' */;
/*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='' */;
/*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;
```



```
--
-- Table structure for table 'a'
--

DROP TABLE IF EXISTS 'a';
/*!40101 SET @saved_cs_client      = @@character_set_client */;
/*!40101 SET character_set_client = utf8 */;
CREATE TABLE 'a' (
  'b' int(11) NOT NULL DEFAULT '0',
  PRIMARY KEY ('b')
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
/*!40101 SET character_set_client = @saved_cs_client */;

/*!40103 SET TIME_ZONE=@OLD_TIME_ZONE */;

/*!40101 SET SQL_MODE=@OLD_SQL_MODE */;
/*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
/*!40101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;
/*!40101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;
/*!40111 SET SQL_NOTES=@OLD_SQL_NOTES */;

-- Dump completed on 2010-08-03 15:36:56
[root@xen-server test]# cat a.txt
1
2
4
5
```

我发现大多数 DBA 喜欢用 SELECT...INTO OUTFILE 的方式来导出一张表，但是通过 mysqldump 一样可以完成工作，而且可以一次完成多张表的导出，并且实现导出数据的一致性。

❑ --where='where_condition' (-w 'where_condition')：导出给定条件的数据。如导出 b 架构下的表 a，并且表 a 的数据大于 2：

```
[root@xen-server bin]# mysqldump --single-transaction --where='b>2' test a >
a.sql
```

```
[root@xen-server bin]# cat a.sql
-- MySQL dump 10.13  Distrib 5.5.1-m2, for unknown-linux-gnu (x86_64)
--
-- Host: localhost    Database: test
```

```

-----
-- Server version          5.5.1-m2-log
.....

--
-- Dumping data for table 'a'
--
-- WHERE:  b>2

LOCK TABLES 'a' WRITE;
/*!40000 ALTER TABLE 'a' DISABLE KEYS */;
INSERT INTO 'a' VALUES (4),(5);
/*!40000 ALTER TABLE 'a' ENABLE KEYS */;
UNLOCK TABLES;
/*!40103 SET TIME_ZONE=@OLD_TIME_ZONE */;
.....

```

8.3.2 SELECT...INTO OUTFILE

SELECT...INTO 语句也是一种逻辑备份的方法，更准确地说是导出一张表中的数据。SELECT...INTO 的语法如下：

```

SELECT [column 1],[column 2] ...
INTO
OUTFILE 'file_name'
[{FIELDS | COLUMNS}
[TERMINATED BY 'string']
[[OPTIONALLY] ENCLOSED BY 'char']
[ESCAPED BY 'char']
]
[LINES
[STARTING BY 'string']
[TERMINATED BY 'string']
]
FROM TABLE WHERE .....

```

其中 FIELDS[TERMINATED BY 'string'] 表示每个列的分隔符，[[OPTIONALLY] ENCLOSED BY 'char'] 表示对于字符串的包含符，[ESCAPED BY 'char'] 表示转义符。[STARTING BY 'string'] 表示每行的开始符号，TERMINATED BY 'string' 表示每行的结束符号。如果没有指定任何的 FIELDS 和 LINES 的选项，默认使用以下的设置：

```

FIELDS TERMINATED BY '\t' ENCLOSED BY '' ESCAPED BY '\\'
LINES TERMINATED BY '\n' STARTING BY ''

```


file_name 表示导出的文件，但文件所在的路径的权限必须是 mysql : mysql 的，否则 MySQL 会报没有权限导出：

```
mysql> select * into outfile '/root/a.txt' from a;
ERROR 1 (HY000): Can't create/write to file '/root/a.txt' (Errcode: 13)
```

若已经存在该文件，则同样会报错：

```
[root@xen-server ~]# mysql test -e "select * into outfile '/home/mysql/a.txt'
fields terminated by ',' from a";
ERROR 1086 (HY000) at line 1: File '/home/mysql/a.txt' already exists
```

查看通过 SELECT INTO 导出的表 a 文件：

```
mysql> select * into outfile '/home/mysql/a.txt' from a;
Query OK, 3 rows affected (0.02 sec)
```

```
mysql> quit
Bye
[root@xen-server ~]# cat /home/mysql/a.txt
1      a
2      b
3      c
```

可以发现，默认导出的文件是以 TAB 进行列分割的，如果想要使用其他分割符，如“，”，则可以使用 FIELDS TERMINATED BY 'string' 选项，如：

```
[root@xen-server ~]# mysql test -e "select * into outfile '/home/mysql/a.txt'
fields terminated by ',' from a";
[root@xen-server ~]# cat /home/mysql/a.txt
1,a
2,b
3,c
```

在 Windows 平台下，由于换行符是“\r\n”，因此在导出时可能需要指定 LINES TERMINATED BY 选项，如：

```
[root@xen-servermysql]# mysql test -e "select * into outfile '/home/mysql/a.txt'
fields terminated by ',' lines terminated by '\r\n' from a";

[root@xen-servermysql]# od -c a.txt
0000000  1  ,  a  \r  \n  2  ,  b  \r  \n  3  ,  c  \r  \n
0000017
```

8.3.3 逻辑备份的恢复

mysqldump 的恢复操作比较简单, 因为备份的文件就是导出的 SQL 语句, 一般只需要执行这个文件就可以了, 可以通过以下的方法:

```
[root@xen-server ~]# mysql -uroot -p <test_backup.sql
Enter password:
```

如果在导出时包含了创建和删除数据库的 SQL 语句, 那必须确保删除架构时, 架构目录下没有其他与数据库相关的文件, 否则可能会得到以下的错误:

```
mysql> drop database test;
ERROR 1010 (HY000): Error dropping database (can't rmdir './test', errno: 39)
```

因为逻辑备份的文件是由 SQL 语句组成的, 也可以通过 SOURCE 命令来执行导出的逻辑备份文件, 如下:

```
mysql> source /home/mysql/test_backup.sql;
Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

.....

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

通过 mysqldump 可以恢复数据库, 但是经常发生的一个问题是, mysqldump 可以导出存储过程、导出触发器、导出事件、导出数据, 但是却不能导出视图。因此, 如果用户的数据库中还使用了视图, 那么在用 mysqldump 备份完数据库后还需要导出视图的定义, 或者备份视图定义的 frm 文件, 并在恢复时进行导入, 这样才能保证 mysqldump 数据库的完全恢复。

8.3.4 LOAD DATA INFILE

若通过 mysqldump-tab, 或者通过 SELECT INTO OUTFILE 导出的数据需要恢复, 这时可以通过命令 LOAD DATA INFILE 来进行导入。LOAD DATA INFILE 的语法如下:

```
LOAD DATA INTO TABLE a IGNORE 1 LINES INFILE '/home/mysql/a.txt'
```

```

[REPLACE | IGNORE]
INTO TABLE tbl_name
[CHARACTER SET charset_name]
[{FIELDS | COLUMNS}
[TERMINATED BY 'string'
[[OPTIONALLY] ENCLOSED BY 'char'
[ESCAPED BY 'char'
]
[LINES
[STARTING BY 'string'
[TERMINATED BY 'string'
]
[IGNORE number LINES]
[(col_name_or_user_var,...)]
[SET col_name= expr,...]

```

要对服务器文件使用 LOAD DATA INFILE，必须拥有 FILE 权。其中对于导入格式的选项和之前介绍的 SELECT INTO OUTFILE 命令完全一样。IGNORE number LINES 选项可以忽略导入的前几行。下面显示一个用 LOAD DATA INFILE 命令导入文件的示例，并忽略第一行的导入：

```

mysql> load data infile '/home/mysql/a.txt' into table a;
Query OK, 3 rows affected (0.00 sec)
Records: 3 Deleted: 0 Skipped: 0 Warnings: 0

```

为了加快 InnoDB 存储引擎的导入，可能希望导入过程忽略对外键的检查，因此可以使用如下方式：

```

mysql>SET @@foreign_key_checks=0;
Query OK, 0 rows affected (0.00 sec)

mysql>LOAD DATA INFILE '/home/mysql/a.txt' INTO TABLE a;
Query OK, 4 rows affected (0.00 sec)
Records: 4 Deleted: 0 Skipped: 0 Warnings: 0

mysql>SET @@foreign_key_checks=1;
Query OK, 0 rows affected (0.00 sec)

```

另外可以针对指定的列进行导入，如将数据导入列 a、b，而 c 列等于 a、b 列之和：

```

mysql>CREATE TABLE b (
    ->a INT,
    ->b INT,
    ->c INT,

```



```

-> PRIMARY KEY(a)
->)ENGINE=InnoDB;
Query OK, 0 rows affected (0.01 sec)

mysql>LOAD DATA INFILE '/home/mysql/a.txt'
->INTO TABLE b FIELDS TERMINATED BY ',' (a,b)
• -> SET c=a+b;
Query OK, 4 rows affected (0.01 sec)
Records: 4 Deleted: 0 Skipped: 0 Warnings: 0

mysql>SELECT * FROM b;
+----+-----+-----+
| a | b | c |
+----+-----+-----+
| 1 | 2 | 3 |
| 2 | 3 | 5 |
| 4 | 5 | 9 |
| 5 | 6 | 11 |
+----+-----+-----+
4 rows in set (0.00 sec)

```

8.3.5 mysqlimport

mysqlimport 是 MySQL 数据库提供的一个命令行程序，从本质上来说，是 LOAD DATA INFILE 的命令接口，而且大多数的选项都和 LOAD DATA INFILE 语法相同。其语法格式如下：

```
shell>mysqlimport [options] db_name textfile1 [textfile2 ...]
```

和 LOAD DATA INFILE 不同的是，mysqlimport 命令可以用来导入多张表。并且通过 --user-thread 参数并发地导入不同的文件。这里的并发是指并发导入多个文件，而不是指 mysqlimport 可以并发地导入一个文件，这是有明显区别的。此外，通常来说并发地对同一张表进行导入，其效果一般都不会比串行的方式好。下面通过 mysqlimport 并发地导入 2 张表：

```

[root@xen-servermysql]# mysqlimport --use-threads=2 test /home/mysql/t.txt /
home/mysql/s.txt
test.s: Records: 5000000 Deleted: 0 Skipped: 0 Warnings: 0
test.t: Records: 5000000 Deleted: 0 Skipped: 0 Warnings: 0

```

如果在上述命令的运行过程中，查看 MySQL 的数据库线程列表，应该可以看到类

似如下内容:

```
mysql>SHOW FULL PROCESSLIST\G;
***** 1. row *****
      Id: 46
     User: rep
    Host: www.dao.com:1028
       db: NULL
 Command: Binlog Dump
      Time: 37651
    State: Master has sent all binlog to slave; waiting for binlog to be updated
      Info: NULL
***** 2. row *****
      Id: 77
     User: root
    Host: localhost
       db: test
 Command: Query
      Time: 0
    State: NULL
      Info: show full processlist
***** 3. row *****
      Id: 83
     User: root
    Host: localhost
       db: test
 Command: Query
      Time: 73
    State: NULL
      Info: LOAD DATA INFILE '/home/mysql/t.txt' INTO TABLE 't' IGNORE 0 LINES
***** 4. row *****
      Id: 84
     User: root
    Host: localhost
       db: test
 Command: Query
      Time: 73
    State: NULL
      Info: LOAD DATA INFILE '/home/mysql/s.txt' INTO TABLE 's' IGNORE 0 LINES
4 rows in set (0.00 sec)
```

可以看到 mysqlimport 实际上是同时执行了两句 LOAD DATA INFILE 并发地导入数据。

8.4 二进制日志备份与恢复

二进制日志非常关键，用户可以通过它完成 point-in-time 的恢复工作。MySQL 数据库的 replication 同样需要二进制日志。在默认情况下并不启用二进制日志，要使用二进制日志首先必须启用它。如在配置文件中设置：

```
[mysqld]
log-bin=mysql-bin
```

在 3.2.4 节中已经阐述过，对于 InnoDB 存储引擎只简单启用二进制日志是不够的，还需要启用一些其他参数来保证最为安全和正确地记录二进制日志，因此对于 InnoDB 存储引擎，推荐的二进制日志的服务器配置应该是：

```
[mysqld]
log-bin = mysql-bin
sync_binlog = 1
innodb_support_xa = 1
```

在备份二进制日志文件前，可以通过 FLUSH LOGS 命令来生成一个新的二进制日志文件，然后备份之前的二进制日志。

要恢复二进制日志也是非常简单的，通过 mysqlbinlog 即可。mysqlbinlog 的使用方法如下：

```
shell>mysqlbinlog [options] log_file...
```

例如要还原 binlog.0000001，可以使用如下命令：

```
shell>mysqlbinlog binlog.0000001 | mysql-uroot -p test
```

如果需要恢复多个二进制日志文件，最正确的做法应该是同时恢复多个二进制日志文件，而不是一个一个地恢复，如：

```
shell>mysqlbinlog binlog.[0-10]* | mysql -u root -p test
```

也可以先通过 mysqlbinlog 命令导出到一个文件，然后再通过 SOURCE 命令来导入，这种做法的好处是可以对导出的文件进行修改后再导入，如：

```
shell>mysqlbinlog binlog.0000001 > /tmp/statements.sql
shell>mysqlbinlog binlog.0000002 >> /tmp/statements.sql
shell>mysql -u root -p -e "source /tmp/statements.sql"
```

--start-position 和 --stop-position 选项可以用来指定从二进制日志的某个偏移量来进

行恢复，这样可以跳过某些不正确的语句，如：

```
shell>mysqlbinlog--start-position=107856 binlog.0000001 | mysql-uroot -p test
```

--start-datetime 和 --stop-datetime 选项可以用来指定从二进制日志的某个时间点来进行恢复，用法和 --start-position 和 --stop-position 选项基本相同。

8.5 热备

8.5.1 ibbackup

ibbackup 是 InnoDB 存储引擎官方提供的热备工具，可以同时备份 MyISAM 存储引擎和 InnoDB 存储引擎表。对于 InnoDB 存储引擎表其备份工作原理如下：

- 1) 记录备份开始时，InnoDB 存储引擎重做日志文件检查点的 LSN。
- 2) 复制共享表空间文件以及独立表空间文件。
- 3) 记录复制完表空间文件后，InnoDB 存储引擎重做日志文件检查点的 LSN。
- 4) 复制在备份时产生的重做日志。

对于事务的数据库，如 Microsoft SQL Server 数据库和 Oracle 数据库，热备的原理大致和上述相同。可以发现，在备份期间不会对数据库本身有任何影响，所做的操作只是复制数据库文件，因此任何对数据库的操作都是允许的，不会阻塞任何操作。故 ibbackup 的优点如下：

- ☐ 在线备份，不阻塞任何的 SQL 语句。
- ☐ 备份性能好，备份的实质是复制数据库文件和重做日志文件。
- ☐ 支持压缩备份，通过选项，可以支持不同级别的压缩。
- ☐ 跨平台支持，ibbackup 可以运行在 Linux、Windows 以及主流的 UNIX 系统平台上。

ibbackup 对 InnoDB 存储引擎表的恢复步骤为：

- ☐ 恢复表空间文件。
- ☐ 应用重做日志文件。

ibbackup 提供了一种高性能的热备方式，是 InnoDB 存储引擎备份的首选方式。不过它是收费软件，并非免费的软件。好在开源的魅力就在于社区的力量，Percona 公司给用户带来了开源、免费的 XtraBackup 热备工具，它实现所有 ibbackup 的功能，并且扩展支

持了真正的增量备份功能。因此,更好的选择是使用 XtraBackup 来完成热备的工作。

8.5.2 XtraBackup

XtraBackup 备份工具是由 Percona 公司开发的开源热备工具。支持 MySQL5.0 以上的版本。XtraBackup 在 GPL v2 开源下发布,官网地址是: <https://launchpad.net/percona-xtrabackup>。

xtrabackup 命令的使用方法如下:

```
xtrabackup--backup | --prepare [OPTIONS]
```

xtrabackup 命令的可选参数如下:

```
(The defaults options should be given as the first argument)
--print-defaults          Prints the program's argument list and exit.
--no-defaults             Don't read the default options from any file.
--defaults-file=          Read the default options from this file.
--defaults-extra-file=    Read this file after the global options files have been
read.
--target-dir=             The destination directory for backups.
--backup                  Make a backup of a mysql instance.
--stats                   Calculate the statistic of the datadir (it is recommended
you take mysqld offline).
--prepare                 Prepare a backup so you can start mysql server with your
restore.
--export                  Create files to import to another database after it has been
prepared.
--print-param             Print the parameters of mysqld that you will need for a
forcopyback.
--use-memory=            This value is used instead of buffer_pool_size.
--suspend-at-end          Creates a file called xtrabackup_suspended and waits until
the user deletes that file at the end of the backup.
--throttle=              (use with --backup) Limits the IO operations (pairs of reads
and writes) per second to the values set here.
--log-stream              outputs the contents of the xtrabackup_logfile to stdout.
--incremental-lsn=       (use with --backup) Copy only .ibd pages newer than the
specified LSN high:low.
                        ##ATTENTION##: checkpoint lsn *must* be used. Be Careful!
--incremental-basedir=   (use with --backup) Copy only .ibd pages newer than
                        the existing backup at the specified directory.
--incremental-dir=       (use with --prepare) Apply .delta files and logfiles
located in the specified directory.
```

```
--tables=name          Regular Expression list of table names to be backed up.
--create-ib-logfile     (NOT CURRENTLY IMPLEMENTED) will create ib_logfile* after
a --prepare.
```

```
### If you want to create ib_logfile* only re-execute this
command using the same options. ###
```

```
--datadir=name          Path to the database root.
--tmpdir=name           Path for temporary files. Several paths may be specified
as a colon (:) separated string.
```

If you specify multiple paths they are used round-robin.

如果用户要做一个完全备份，可以执行如下命令：

```
# ./xtrabackup --backup
./xtrabackup Ver alpha-0.2 for 5.0.75 unknown-linux-gnu (x86_64)
>>log scanned up to (0 1009910580)
Copying ./ibdata1
to /home/kinoyasu/xtrabackup_work/mysql-5.0.75/innobase/xtrabackup/tmp2/ibdata1
...done
Copying ./tpcc/stock.ibd
to /home/kinoyasu/xtrabackup_work/mysql-5.0.75/innobase/xtrabackup/tmp2/tpcc/
stock.ibd
...done
Copying ./tpcc/new_orders.ibd
to /home/kinoyasu/xtrabackup_work/mysql-5.0.75/innobase/xtrabackup/tmp2/tpcc/
new_orders.ibd
...done
Copying ./tpcc/history.ibd
to /home/kinoyasu/xtrabackup_work/mysql-5.0.75/innobase/xtrabackup/tmp2/tpcc/
history.ibd
...done
Copying ./tpcc/customer.ibd
to /home/kinoyasu/xtrabackup_work/mysql-5.0.75/innobase/xtrabackup/tmp2/tpcc/
customer.ibd
>>log scanned up to (0 1010561109)
...done
Copying ./tpcc/district.ibd
to /home/kinoyasu/xtrabackup_work/mysql-5.0.75/innobase/xtrabackup/tmp2/tpcc/
district.ibd
...done
Copying ./tpcc/item.ibd
to /home/kinoyasu/xtrabackup_work/mysql-5.0.75/innobase/xtrabackup/tmp2/tpcc/
item.ibd
...done
Copying ./tpcc/order_line.ibd
```



```

    to /home/kinoyasu/xtrabackup_work/mysql-5.0.75/innobase/xtrabackup/tmp2/tpcc/
order_line.ibd
    >>log scanned up to (0 1012047066)
        ...done
    Copying ./tpcc/orders.ibd
    to /home/kinoyasu/xtrabackup_work/mysql-5.0.75/innobase/xtrabackup/tmp2/tpcc/
orders.ibd
        ...done
    Copying ./tpcc/warehouse.ibd
    to /home/kinoyasu/xtrabackup_work/mysql-5.0.75/innobase/xtrabackup/tmp2/tpcc/
warehouse.ibd
        ...done
    >>log scanned up to (0 1014592707)
    Stopping log copying thread..
Transaction log of lsn (0 1009910580) to (0 1014592707) was copied.

```

可以看到在开始备份时，xtrabackup 首先记录了重做日志的位置，在上述示例中为 (0 1009910580)。然后对备份的 InnoDB 存储引擎表的物理文件，即共享表空间和独立表空间进行 copy 操作，这里可以看到输出有 Copying...to...。最后记录备份完成后的重做日志位置 (0 1014592707)。

8.5.3 XtraBackup 实现增量备份

MySQL 数据库本身提供的工具并不支持真正的增量备份，更准确地说，二进制日志的恢复应该是 point-in-time 的恢复而不是增量备份。而 XtraBackup 工具支持对于 InnoDB 存储引擎的增量备份，其工作原理如下：

- 1) 首先完成一个全备，并记录下此时检查点的 LSN。
- 2) 在进行增量备份时，比较表空间中每个页的 LSN 是否大于上次备份时的 LSN，如果是，则备份该页，同时记录当前检查点的 LSN。

因此 XtraBackup 的备份和恢复的过程大致如下：

```

(full backup)
# ./xtrabackup --backup --target-dir=/backup/base
...

(incremental backup)
# ./xtrabackup --backup --target-dir=/backup/delta --incremental-basedir=/
backup/base
...

```



```
(prepare)
# ./xtrabackup --prepare --target-dir=/backup/base
...

(apply incremental backup)
# ./xtrabackup --prepare --target-dir=/backup/base --incremental-dir=/backup/
delta
...
```

在上述过程中，首先将全部文件备份到 /backup/base 目录下，增量备份产生的文件备份到 /backup/delta。在恢复过程中，首先指定全备的路径，然后将增量的备份应用于该完全备份。以下显示了一个完整的增量备份过程：

```
# ./xtrabackup --backup
./xtrabackup Ver beta-0.4 for 5.0.75 unknown-linux-gnu (x86_64)
>>log scanned up to (0 378161500)
...
The latest check point (for incremental): '0:377883685'    <===== 使用这个 LSN
>>log scanned up to (0 379294296)
Stopping log copying thread..
Transaction log of lsn (0 377883685) to (0 379294296) was copied.

(must do --prepare before the each incremental backup)
# ./xtrabackup --prepare
...

# ./xtrabackup --backup --incremental=0:377883685
incremental backup from 0:377883685 is enabled.
./xtrabackup Ver beta-0.4 for 5.0.75 unknown-linux-gnu (x86_64)
>>log scanned up to (0 379708047)
Copying ./ibdata1
to /home/kinoyasu/xtrabackup_work/mysql-5.0.75/innobase/xtrabackup/tmp_diff/
ibdata1.delta
...done
...

The latest check point (for incremental): '0:379438233'    <===== 下一个增量备份开
始的 LSN
>>log scanned up to (0 380663549)
Stopping log copying thread..
Transaction log of lsn (0 379438233) to (0 380663549) was copied.
```

8.6 快照备份

MySQL 数据库本身并不支持快照功能，因此快照备份是指通过文件系统支持的快照功能对数据库进行备份。备份的前提是将所有数据库文件放在同一文件分区中，然后对该分区进行快照操作。支持快照功能的文件系统和设备包括 FreeBSD 的 UFS 文件系统，Solaris 的 ZFS 文件系统，GNU/Linux 的逻辑管理器（Logical Volume Manager, LVM）等。这里以 LVM 为例进行介绍，UFS 和 ZFS 的快照实现大致和 LVM 相似。

LVM 是 LINUX 系统下对磁盘分区进行管理的一种机制。LVM 在硬盘和分区之上建立一个逻辑层，来提高磁盘分区管理的灵活性。管理员可以通过 LVM 系统轻松管理磁盘分区，例如，将若干个磁盘分区连接为一个整块的卷组（Volume Group），形成一个存储池。管理员可以在卷组上随意创建逻辑卷（Logical Volumes），并进一步在逻辑卷上创建文件系统。管理员通过 LVM 可以方便地调整卷组的大小，并且可以对磁盘存储按照组的方式进行命名、管理和分配。简单地说，用户可以通过 LVM 由物理块设备（如硬盘等）创建物理卷，由一个或多个物理卷创建卷组，最后从卷组中创建任意个逻辑卷（不超过卷组大小），如图 8-1 所示。

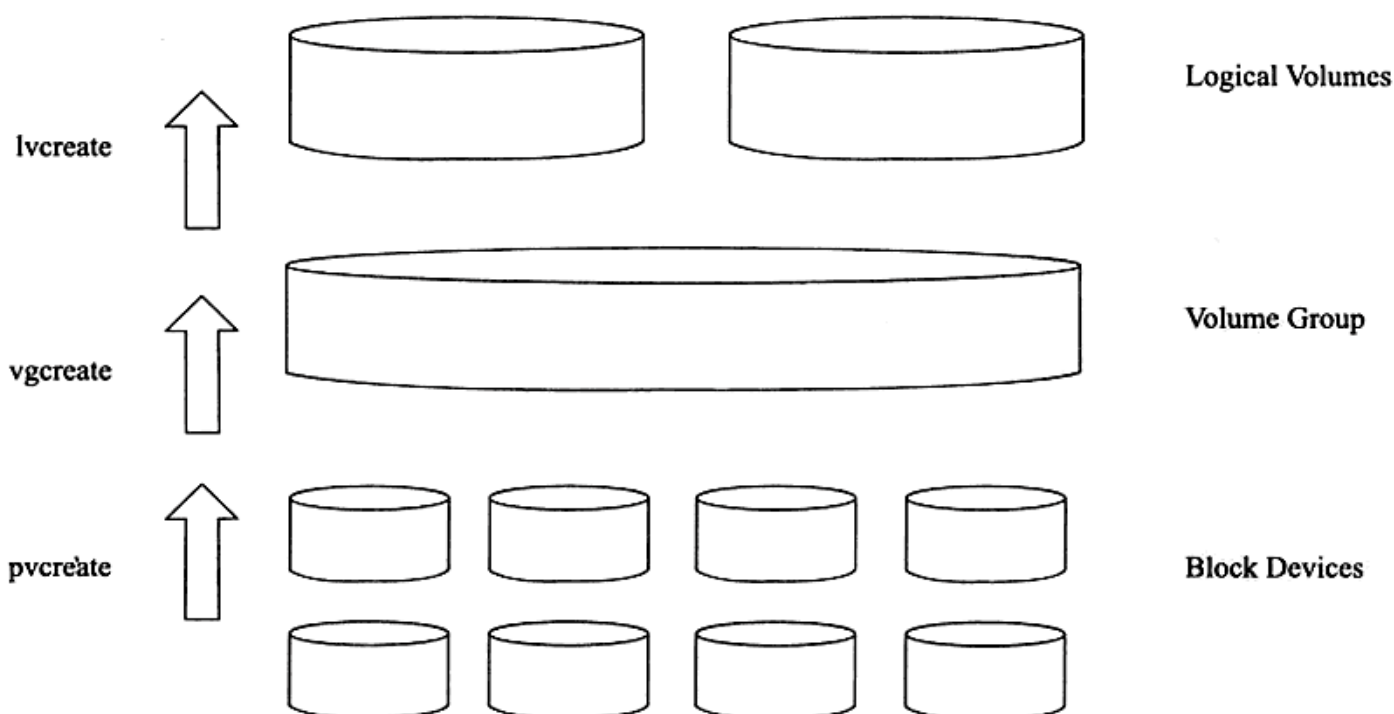


图 8-1 LVM 工作原理

图 8-2 显示了由多块磁盘组成的逻辑卷 LV0。

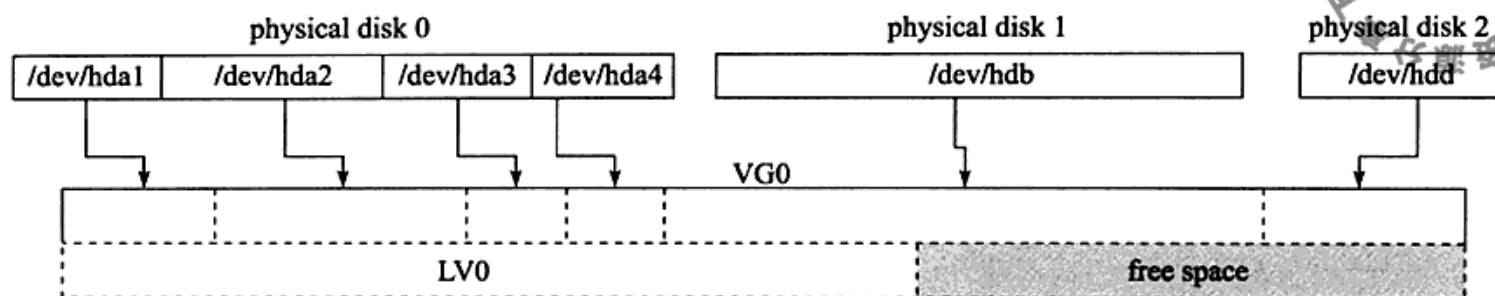


图 8-2 物理到逻辑卷的映射

通过 `vgdisplay` 命令查看系统中有哪些卷组，如：

```
[root@nhl24-98 ~]# vgdisplay
--- Volume group ---
VG Name                rep
System ID
Format                 lvm2
Metadata Areas         1
Metadata Sequence No   1873
VG Access               read/write
VG Status               resizable
MAX LV                 0
Cur LV                 3
Open LV                 1
Max PV                 0
Cur PV                 1
Act PV                 1
VG Size                 260.77 GB
PE Size                 4.00 MB
Total PE                66758
Alloc PE / Size        66560 / 260.00 GB
Free PE / Size          198 / 792.00 MB
VG UUID                 MQJiye-j4NN-LbZG-F3CQ-UdTU-fo9D-RRfXD5
```

`vgdisplay` 命令的输出结果显示当前系统有一个 `rep` 的卷组，大小为 260.77GB，该卷组访问权限是 `read/write` 等。命令 `lvdisplay` 可以用来查看当前系统中有哪些逻辑卷：

```
[root@nhl24-98 ~]# lvdisplay
--- Logical volume ---
LV Name                /dev/rep/repdata
VG Name                rep
LV UUID                 7tOlDt-seKZ-ChpY-QMXC-WaFD-zXA1-MRbofK
LV Write Access         read/write
LV snapshot status      source of
                        /dev/rep/dho_datasnapshot100805143507 [active]
                        /dev/rep/dho_datasnapshot100805163504 [active]
```

```

LV Status          available
# open             1
LV Size            100.00 GB
Current LE         25600
Segments           1
Allocation          inherit
Read ahead sectors auto
- currently set to 256
Block device       253:0

```

--- Logical volume ---

```

LV Name            /dev/rep/dho_datasnapshot100805143507
VG Name            rep
LV UUID            fSSXzh-IBnZ-aZIn-eP03-b7pk-CPjN-5xUktE
LV Write Access    read only
LV snapshot status active destination for /dev/rep/repdata
LV Status          available
# open             0
LV Size            100.00 GB
Current LE         25600
COW-table size     80.00 GB
COW-table LE       20480
Allocated to snapshot 0.13%
Snapshot chunk size 4.00 KB
Segments           1
Allocation          inherit
Read ahead sectors auto
- currently set to 256
Block device       253:1

```

--- Logical volume ---

```

LV Name            /dev/rep/dho_datasnapshot100805163504
VG Name            rep
LV UUID            3B9NP1-qWVG-pfJY-Bdgm-DIdD-dUMu-s2L6qJ
LV Write Access    read only
LV snapshot status active destination for /dev/rep/repdata
LV Status          available
# open             0
LV Size            100.00 GB
Current LE         25600
COW-table size     80.00 GB
COW-table LE       20480
Allocated to snapshot 0.02%
Snapshot chunk size 4.00 KB

```

```

Segments          1
Allocation         inherit
Read ahead sectors auto
- currently set to 256
Block device       253:4

```

可以看到，一共有 3 个逻辑卷，都属于卷组 rep，每个逻辑卷的大小都是 100GB。
/dev/rep/repdata 这个逻辑卷有两个只读快照，并且当前都是激活状态的。

LVM 使用了写时复制（Copy-on-write）技术来创建快照。当创建一个快照时，仅复制原始卷中数据的元数据（meta data），并不会对数据的物理操作，因此快照的创建过程是非常快的。当快照创建完成，原始卷上有写操作时，快照会跟踪原始卷块的变化，将要改变的数据在改变之前复制到快照预留的空间里，因此这个原理的实现叫做写时复制。而对于快照的读取操作，如果读取的数据块是创建快照后没有修改过的，那么会将读操作直接重定向到原始卷上，如果要读取的是已经修改过的块，则将读取保存在快照中该块在原始卷上改变之前的数据。因此，采用写时复制机制保证了读取快照时得到的数据与快照创建时一致。

图 8-3 显示了 LVM 的快照读取，可见 B 区块被修改了，因此历史数据放入了快照区域。读取快照数据时，A、C、D 块还是从原有卷中读取，而 B 块就需要从快照读取了。

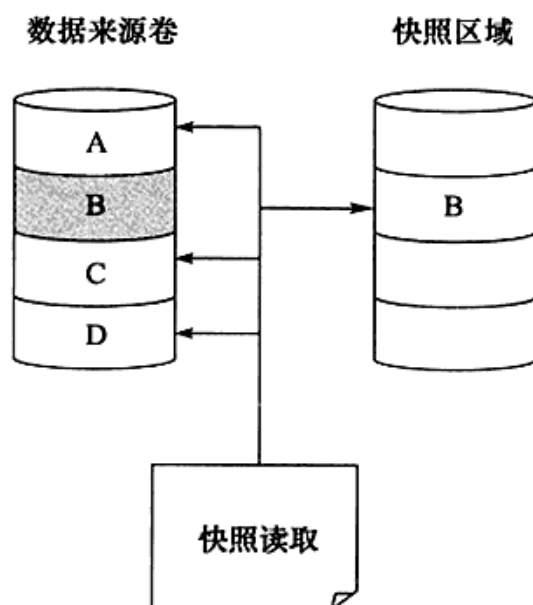


图 8-3 LVM 快照读取

命令 `lvcreate` 可以用来创建一个快照，`--permission r` 表示创建的快照是只读的：

```

[root@nh119-215 data]# lvcreate --size 100G --snapshot --permission r -n
datasnapshot /dev/rep/repdata
Logical volume "datasnapshot" created

```

在快照制作完成后可以用 `lvdisplay` 命令查看，输出中的 `COW-table size` 字段表示该快照最大的空间大小，`Allocated to snapshot` 字段表示该快照目前空间的使用状况：

```

[root@nh124-98 ~]# lvdisplay
.....
--- Logical volume ---
LV Name                /dev/rep/dho_datasnapshot100805163504
VG Name                rep

```



```

LV UUID                3B9NP1-qWVG-pfJY-Bdgm-DIdD-dUMu-s2L6qJ
LV Write Access         read only
LV snapshot status     active destination for /dev/rep/repdata
LV Status               available
# open                  0
LV Size                 100.00 GB
Current LE              25600
COW-table size          80.00 GB
COW-table LE            20480
Allocated to snapshot  0.04%
Snapshot chunk size     4.00 KB
Segments                1
Allocation              inherit
Read ahead sectors      auto
- currently set to      256
Block device            253:4

```

可以看到，当前快照只使用 0.04% 的空间。快照在最初创建时总是很小，当数据来源卷的数据不断被修改时，这些数据库才会放入快照空间，这时快照的大小才会慢慢增大。

用 LVM 快照备份 InnoDB 存储引擎表相当简单，只要把与 InnoDB 存储引擎相关的文件如共享表空间、独立表空间、重做日志文件等放在同一个逻辑卷中，然后对这个逻辑卷做快照备份即可。

在对 InnoDB 存储引擎文件做快照时，数据库无须关闭，即可以进行在线备份。虽然此时数据库中可能还有任务需要往磁盘上写数据，但这不会妨碍备份的正确性。因为 InnoDB 存储引擎是事务安全的引擎，在下次恢复时，数据库会自动检查表空间中页的状态，并决定是否应用重做日志，恢复就好像数据库被意外重启了。

8.7 复制

8.7.1 复制的工作原理

复制（replication）是 MySQL 数据库提供了一种高可用高性能的解决方案，一般用来建立大型的应用。总体来说，replication 的工作原理分为以下 3 个步骤：

- 1) 主服务器（master）把数据更改记录到二进制日志（binlog）中。
- 2) 从服务器（slave）把主服务器的二进制日志复制到自己的中继日志（relay log）中。
- 3) 从服务器重做中继日志中的日志，把更改应用到自己的数据库上，以达到数据

的最终一致性。

复制的工作原理并不复杂，其实就是一个完全备份加上二进制日志备份的还原。不同的是这个二进制日志的还原操作基本上实时在进行中。这里特别需要注意的是，复制不是完全实时地进行同步，而是**异步实时**。这中间存在主从服务器之间的执行延时，如果主服务器的压力很大，则可能导致主从服务器延时较大。复制的工作原理如图 8-4 所示。

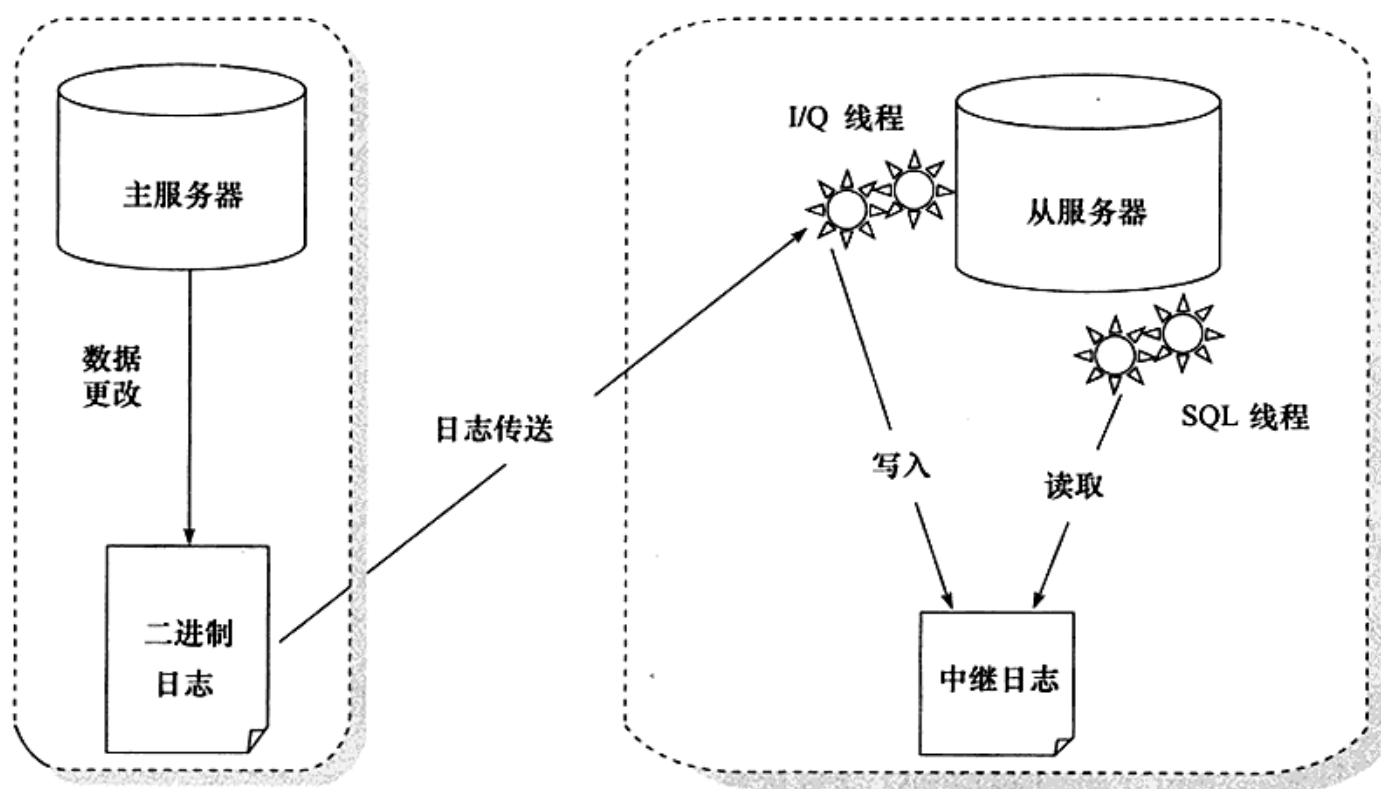


图 8-4 MySQL 数据库的复制工作原理

从服务器有 2 个线程，一个是 I/O 线程，负责读取主服务器的二进制日志，并将其保存为中继日志；另一个是 SQL 线程，复制执行中继日志。MySQL4.0 版本之前，从服务器只有 1 个线程，既负责读取二进制日志，又负责执行二进制日志中的 SQL 语句。这种方式不符合高性能的要求，目前已淘汰。因此如果查看一个从服务器的状态，应该可以看到类似如下内容：

```
mysql>SHOW FULL PROCESSLIST\G;
***** 1. row *****
    Id: 1
  User: system user
   Host:
    db: NULL
Command: Connect
   Time: 6501
  State: Waiting for master to send event
```

```

Info: NULL
***** 2. row *****
    Id: 2
    User: system user
    Host:
    db: NULL
Command: Connect
    Time: 0
    State: Has read all relay log; waiting for the slave I/O thread to update it
    Info: NULL
***** 3. row *****
    Id: 206
    User: root
    Host: localhost
    db: NULL
Command: Query
    Time: 0
    State: NULL
    Info: SHOW FULL PROCESSLIST
3 rows in set (0.00 sec)

```

可以看到 ID 为 1 的线程就是 I/O 线程，目前的状态是等待主服务器发送二进制日志。ID 为 2 的线程是 SQL 线程，负责读取中继日志并执行。目前的状态是已读取所有的中继日志，等待中继日志被 I/O 线程更新。

在 replication 的主服务器上应该可以看到一个线程负责发送二进制日志，类似内容如下：

```

mysql>SHOW FULL PROCESSLIST\G;
.....
***** 65. row *****
    Id: 26541
    User: rep
    Host: 192.168.190.98:39549
    db: NULL
Command: Binlog Dump
    Time: 6857
    State: Has sent all binlog to slave; waiting for binlog to be updated
    Info: NULL
.....

```

之前已经说过 MySQL 的复制是异步实时的，并非完全的主从同步。若用户要想得知当前的延迟，可以通过命令 SHOW SLAVE STATUS 和 SHOW MASTER STATUS 得知，如：



```
mysql>SHOW SLAVE STATUS\G;
***** 1. row *****
      Slave_IO_State: Waiting for master to send event
        Master_Host: 192.168.190.10
        Master_User: rep
        Master_Port: 3306
        Connect_Retry: 60
        Master_Log_File: mysql-bin.000007
    Read_Master_Log_Pos: 555176471
        Relay_Log_File: gamedb-relay-bin.000048
        Relay_Log_Pos: 224355889
    Relay_Master_Log_File: mysql-bin.000007
      Slave_IO_Running: Yes
     Slave_SQL_Running: Yes
        Replicate_Do_DB:
    Replicate_Ignore_DB:
        Replicate_Do_Table:
    Replicate_Ignore_Table:
    Replicate_Wild_Do_Table:
    Replicate_Wild_Ignore_Table: mysql.%,DBA.%
          Last_Errno: 0
          Last_Error:
        Skip_Counter: 0
    Exec_Master_Log_Pos: 555176471
        Relay_Log_Space: 224356045
        Until_Condition: None
        Until_Log_File:
        Until_Log_Pos: 0
    Master_SSL_Allowed: No
    Master_SSL_CA_File:
        Master_SSL_CA_Path:
        Master_SSL_Cert:
        Master_SSL_Cipher:
        Master_SSL_Key:
    Seconds_Behind_Master: 0
Master_SSL_Verify_Server_Cert: No
          Last_IO_Errno: 0
          Last_IO_Error:
        Last_SQL_Errno: 0
        Last_SQL_Error:
1 row in set (0.00 sec)
```

通过 SHOW SLAVE STATUS 命令可以观察当前复制的运行状态，一些主要的变量

如表 8-1 所示。

表 8-1 SHOW SLAVE STATUS 的主要变量

变 量	说 明
Slave_IO_State	显示当前 IO 线程的状态，上述状态显示的是等待主服务发送二进制日志
Master_Log_File	显示当前同步的主服务器的二进制日志，上述显示当前同步的是主服务器的 mysql-bin.000007
Read_Master_Log_Pos	显示当前同步到主服务器上二进制日志的偏移量位置，单位是字节。上述的示例显示当前同步到 mysql-bin.000007 的 555176471 偏移量位置，即已经同步了 mysql-bin.000007 这个二进制日志中 529MB (555176471/1024/1024) 的内容
Relay_Master_Log_File	当前中继日志同步的二进制日志
Relay_Log_File	显示当前写入的中继日志
Relay_Log_Pos	显示当前执行到中继日志的偏移量位置
Slave_IO_Running	从服务器中 IO 线程的运行状态，YES 表示运行正常
Slave_SQL_Running	从服务器中 SQL 线程的运行状态，YES 表示运行正常
Exec_Master_Log_Pos	表示同步到主服务器的二进制日志偏移量的位置。(Read_Master_Log_Pos - Exec_Master_Log_Pos) 可以表示当前 SQL 线程运行的延时，单位是字节。上述例子显示当前主从服务器是完全同步的

命令 SHOW MASTER STATUS 可以用来查看主服务器中二进制日志的状态，如：

```
mysql>SHOW MASTER STATUS\G;
***** 1. row *****
      File: mysql-bin.000007
      Position: 606181078
      Binlog_Do_DB:
      Binlog_Ignore_DB:
1 row in set (0.01 sec)
```

可以看到，当前二进制日志记录了偏移量 606181078 的位置，该值减去这一时间点时从服务器上的 Read_Master_Log_Pos，就可以得知 I/O 线程的延时。

对于一个优秀的 MySQL 数据库复制的监控，用户不应该仅仅监控从服务器上 I/O 线程和 SQL 线程运行得是否正常，同时也应该监控从服务器和主服务器之间的延迟，确保从服务器上的数据库总是尽可能地接近于主服务器上数据库的状态。

8.7.2 快照 + 复制的备份架构

复制可以用来作为备份，但功能不仅限于备份，其主要功能如下：

- ❑ 数据分布。由于 MySQL 数据库提供的复制并不需要很大的带宽要求，因此可以在不同的数据中心之间实现数据的复制。

- ❑ 读取的负载均衡。通过建立多个从服务器，可将读取平均地分布到这些从服务器中，并且减少了主服务器的压力。一般通过 DNS 的 Round-Robin 和 Linux 的 LVS 功能都可以实现负载均衡。
- ❑ 数据库备份。复制对备份很有帮助，但是从服务器不是备份，不能完全代替备份。
- ❑ 高可用性和故障转移。通过复制建立的从服务器有助于故障转移，减少故障的停机时间和恢复时间。

可见，复制的设计不是简简单单用来备份的，并且只是用复制来进行备份是远远不够的。假设当前应用采用了主从的复制架构，从服务器作为备份。这时，一个初级 DBA 执行了误操作，如 DROP DATABASE 或 DROP TABLE，这时从服务器也跟着运行了。这时用户怎样从服务器进行恢复呢？

因此，一个比较好的方法是通过对从服务器上的数据库所在分区做快照，以此来避免误操作对复制造成影响。当发生主服务器上的误操作时，只需要将从服务器上的快照进行恢复，然后再根据二进制日志进行 point-in-time 的恢复即可。因此快照 + 复制的备份架构如图 8-5 所示。

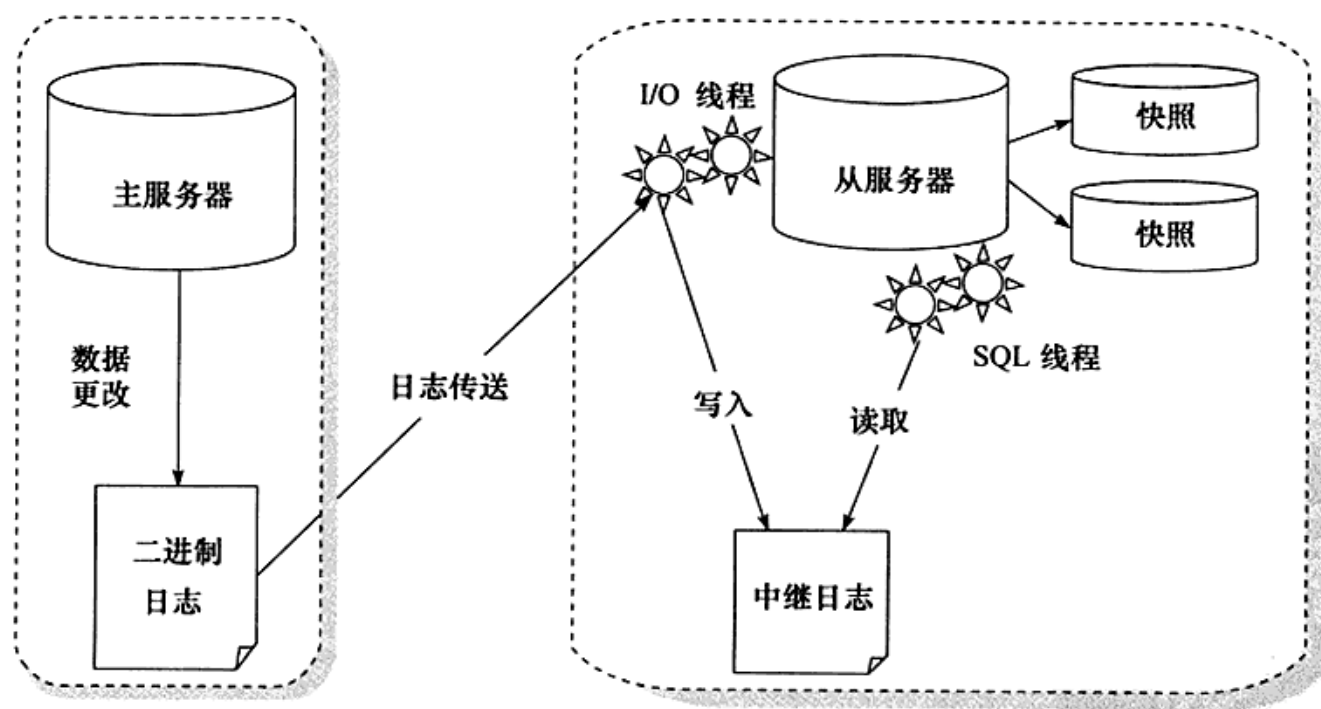


图 8-5 快照 + 复制的备份架构

还有一些其他的方法来调整复制，比如采用延时复制，即间歇性地开启从服务器上的同步，保证大约一小时的延时。这的确也是一个方法，只是数据库在高峰和非高峰期间每小时产生的二进制日志量是不同的，用户很难精准地控制。另外，这种方法也不能



完全起到对误操作的防范作用。

此外，建议在从服务上启用 `read-only` 选项，这样能保证从服务器上的数据仅与主服务器进行同步，避免其他线程修改数据。如：

```
[mysqld]  
read-only
```

在启用 `read-only` 选项后，如果操作从服务器的用户没有 `SUPER` 权限，则对从服务器进行任何的修改操作会抛出一个错误，如：

```
mysql>INSERT INTO z SELECT 2;  
ERROR 1290 (HY000): The MySQL server is running with the --read-only option so  
it cannot execute this statement
```

8.8 小结

本章中介绍了不同的备份类型，并介绍了 MySQL 数据库常用的一些备份方式。同时主要介绍了对于 InnoDB 存储引擎表的备份。不管是 `mysqldump` 还是 `xtrabackup` 工具，都可以对 InnoDB 存储引擎表进行很好的在线热备工作。最后，介绍了复制，通过快照和复制技术的结合，可以保证用户得到一个异步实时的在线 MySQL 备份解决方案。